PROJECT MANUAL

STM32 PULSE COUNTER USING 7-SEGMENT DISPLAY

Objective

To design and implement a digital pulse counter using the STM32F103 microcontroller that counts the number of pulses received through an input pin and displays the count (0–9) on a 7-segment display.

Introduction

Pulse counters are widely used in digital systems to measure the number of occurrences of an event — such as motor revolutions, object detections, or button presses.

In this project, an STM32F103C8 microcontroller is used to detect input pulses (from a button or sensor) and display the count value on a 7-segment display.

The design uses register-level programming (CMSIS) to directly configure and control the GPIO registers for better understanding of hardware-level programming.

Components Required

Component	Specification	Quantity
STM32F103C8 (Blue Pill)	32-bit ARM Cortex-M3	1
7-Segment Display	Common Cathode	1
Push Button	Normally Open (NO)	1
Resistors	220Ω (current limiting for LEDs)	8
Breadboard	–	1
Jumper Wires	Male-to-Male	As required
Power Supply	3.3V	1

Working Principle

1. A push button is connected to GPIOB pin 0 configured as input with an internal pull-up.
2. When the button is pressed, it generates a low pulse (0).
3. Each press increments the pulse counter.
4. The counter value (0–9) is used as an index into a lookup table (seg_code[]) that holds the binary representation for displaying digits on the 7-segment display.
5. The 7-segment display connected to GPIOA shows the count in real time.
6. Once the count reaches 9, it resets to 0.

Pin Configuration

STM32 Pin	Direction	Function	Connected To
PA0 – PA6	Output	7-segment LED segments (a–g)	Segment pins through 220Ω resistors
PB0	Input	Pulse input	Push button
VCC (3.3V)	–	Power	Common 3.3V
GND	–	Ground	Common ground

Circuit Diagram (Text Description)

Connections for 7-segment (Common Cathode):

STM32 PA0 → Segment a
STM32 PA1 → Segment b
STM32 PA2 → Segment c
STM32 PA3 → Segment d
STM32 PA4 → Segment e
STM32 PA5 → Segment f
STM32 PA6 → Segment g
Common Cathode → GND
Each segment → 220Ω resistor → STM32 pin

Push Button Connection:

One side of button → GND

Other side → PB0 (STM32)
PB0 internally pulled up to 3.3V

Source Code (CMSIS-Based)

```
#include "stm32f1xx.h"

int main(void){
    uint8_t seg_code[10] = {
        0x3F, // 0 -> 0b00111111
        0x06, // 1 -> 0b00000110
        0x5B, // 2 -> 0b01011011
        0x4F, // 3 -> 0b01001111
        0x66, // 4 -> 0b01100110
        0x6D, // 5 -> 0b01101101
        0x7D, // 6 -> 0b01111101
        0x07, // 7 -> 0b00000111
        0x7F, // 8 -> 0b01111111
        0x6F  // 9 -> 0b01101111
    };

    int pulse_count = 0;

    RCC->APB2ENR |= RCC_APB2ENR_IOPAEN;
    RCC->APB2ENR |= RCC_APB2ENR_IOPBEN;

    GPIOA->CRL = 0x22222222;
    GPIOB->CRL &= ~(0xF << 0);
    GPIOB->CRL |=  (0x8 << 0);
    GPIOB->ODR |=  (0x1 << 0);

    GPIOA->ODR = 0x00;
    GPIOA->ODR = seg_code[pulse_count];

    while(1){
        if ((GPIOB->IDR & (1 << 0)) == 0){
            while(!(GPIOB->IDR & (1<<0)));
```

```
            pulse_count = pulse_count + 1;
            if (pulse_count == 10){
                pulse_count = 0;
            }
            GPIOA->ODR = 0x00;
            GPIOA->ODR = seg_code[pulse_count];
        }
    }

}
```

Explanation of Code

Step	Code Line	Function
1	RCC->APB2ENR	Enables peripheral clocks for GPIO ports
2	GPIOA->CRL	Configures PA0–PA7 as output
3	GPIOB->CRL	Configures PB0 as input with pull-up
4	GPIOB->ODR	Sets pull-up resistor active
5	seg_code[]	Holds binary codes for displaying digits 0–9
6	if ((GPIOB->IDR & (1<<0)) == 0)	Checks for button press
7	while (!(GPIOB->IDR & (1<<0)));	Debounces and waits for release
8	pulse_count++	Increments counter
9	GPIOA->ODR = seg_code[pulse_count];	Updates 7-segment display

---

Testing Procedure

1. Connect all components as per the pin configuration.
2. Flash the program into the STM32 board using ST-Link Utility or STM32CubeProgrammer.

3. Apply 3.3V power supply to the board.
4. Observe the display shows 0 initially.
5. Press the button once — display changes to 1.
6. Continue pressing to increment count up to 9.
7. After 9, it rolls back to 0.

Observations

Number of Button Presses	Display Output
0	0
1	1
2	2
3	3
4	4
5	5
6	6
7	7
8	8
9	9
10	0 (reset)

Simulation in Proteus

1. Place STM32F103C8T6 from the Proteus component library.
2. Add 7-segment display (Common Cathode) and Push Button.
3. Wire PA0–PA6 → Segments (a–g).
4. Connect PB0 → Push Button (to GND).
5. Add virtual terminal (optional) to debug count.
6. Load the compiled .hex file into STM32.
7. Run the simulation and press the button — observe digit change.

Advantages

- Simple and low-cost implementation.
- Teaches direct register manipulation.
- Good example for interrupt-less polling method.
- Easy to extend for dual 7-segment (00–99) counters.

Applications

- Event counting (sensors, motors).
- Digital product counters.
- Frequency and speed measurement.
- Basic lab training on GPIO handling.

Result

When the push button is pressed, the 7-segment display increments from 0 → 9, then resets back to 0.
This confirms successful counting and display functionality.

Conclusion

The project successfully demonstrates how to interface a 7-segment display with the STM32F103 microcontroller and count input pulses.
This provides a strong foundation for building more advanced event counters and digital measurement systems.
